

AutoSec-AI

A Policy-Gated Agentic AI Framework
for Automotive Cybersecurity Assessment

Engineering Prototype

Architecture, Security Design and Evaluation

Author: Trivendra Kulhare

Domain: Automotive Cybersecurity / Agentic AI

Report Type: Technical Engineering Report

Date: September 2026

*AutoSec-AI is an independent engineering prototype developed for controlled cybersecurity research
and technical learning.*

<https://github.com/trivendrakulhare/AutoSec-AI>

Abstract

I built AutoSec-AI to explore a practical question that becomes important when large language models are connected to cybersecurity tools: how much authority should the model actually have?

An LLM can be useful for planning an investigation, interpreting evidence, and deciding what might be worth checking next. I did not want those abilities to mean that model output could directly execute a scanner, select an arbitrary target, or change the limits of an investigation.

AutoSec-AI therefore uses a policy-gated architecture. The model proposes actions, while deterministic application logic decides whether those actions are permitted. This leads to the main design principle used throughout the project:

The model proposes. Deterministic policy decides.

The prototype brings together source-code analysis with Semgrep, limited imported-symbol inspection of compiled binaries, simulated CAN/UDS diagnostic behavior, deterministic UDS fuzzing, normalized security evidence, AI-assisted finding assessment, bounded investigation feedback, and terminal/JSON reporting.

A controlled end-to-end demonstration traces one investigation from an AI-facing proposal through authorization, Semgrep execution, normalized CWE-120 evidence, AI-assisted interpretation, investigation history, and bounded termination.

The project is intentionally limited. It does not communicate with a real vehicle, its binary analysis is not a full reverse-engineering engine, the bundled source rule set is small, and an AI assessment is never treated as proof that a vulnerability is exploitable. The main purpose of the project is to investigate the architecture and security boundaries needed when probabilistic AI reasoning is combined with security-sensitive tools.

Contents

1	Executive Summary	4
2	Motivation and Problem Statement	5
2.1	Agentic AI in Security Toolchains	5
2.2	Core Engineering Problem	5
3	Objectives and Scope	5
4	System Architecture	6
5	Agentic AI and Trust-Boundary Design	7
5.1	LLM Output as a Proposal	7
5.2	Proposal and Authorization	8
5.3	Evidence and Interpretation	8
5.4	Historical Feedback	8
5.5	Bounded Autonomy	8
5.6	Resulting Trust Model	9
6	Security Policy and Authorization	9
6.1	Security Policy	9
6.2	Action Validation	10
6.3	Tool Registry	10
6.4	Target Authorization	10
6.5	Parameter Authorization	11
6.6	Trusted Rule-Pack Resolution	11
6.7	External Process Boundary	11
6.8	Rejection Is a Normal Outcome	12
7	Source-Code Security Analysis	12
7.1	Analyzer Abstraction	12
7.2	Rule-Pack Design	12
7.3	Controlled ECU Fixture	13
7.4	Tool Integration	13
7.5	Current Processing Limitation	13
7.6	What the Finding Means	13
8	Binary Security Analysis	14
8.1	Imported-Symbol Inspection	14
8.2	Meaning of the Evidence	14
8.3	Normalized Binary Evidence	14
9	Automotive CAN/UDS Security Simulation	15
9.1	Automotive Data Model	15
9.2	Simulated ECU	15
9.3	Diagnostic Access Control	15
9.4	Evidence Path	16
9.5	Simulation Boundary	16
10	Deterministic UDS Fuzzing	16
10.1	Mutation Cases	17

10.2 Bounded Execution	17
10.3 Generation and Analysis Are Separate	17
10.4 Why Start with Deterministic Fuzzing?	17
11 Evidence Normalization and AI Assessment	18
11.1 SecurityFinding	18
11.2 NormalizedEvidence and FindingContext	18
11.3 AI-Assisted Assessment	18
11.4 Evidence, Assessment, and Authority	19
11.5 Prompt-Injection Considerations	19
12 Closed-Loop Investigation	19
12.1 Investigation State	19
12.2 One Orchestration Step	20
12.3 Bounded Runner	20
12.4 Feedback Does Not Increase Authority	20
13 End-to-End Automotive Source Investigation	20
13.1 Composition	21
13.2 Execution	21
13.3 Representative Output	22
13.4 What This Demonstrates	22
14 Security Evaluation and Testing	22
14.1 Automated Test Suite	23
14.2 Investigation Evaluation	23
14.3 Security-Oriented Testing	24
15 Threat Model and Security Invariants	24
15.1 Protected Assets	24
15.2 Threats and Controls	25
15.3 Security Invariants	25
15.4 Residual Risk	25
16 Limitations	26
17 Future Work	26
17.1 Source Analysis	26
17.2 Binary Analysis	27
17.3 Controlled Vehicle Interfaces	27
17.4 Fuzzing	27
17.5 Execution Isolation	27
17.6 Analyst Interface	28
17.7 Evaluation	28
18 Conclusion	28
A Prototype Environment	29
B Reproducible Demonstration	29
C Project Repository	30

1 Executive Summary

AutoSec-AI started from a simple concern: connecting an LLM to security tools is useful, but allowing the LLM to become the execution authority creates a new security problem.

In a conventional automated workflow, the sequence of operations is usually predetermined. An agentic workflow is different. A planner can look at an objective, consider previous observations, and propose what to do next. That flexibility is valuable for security investigations, where the next useful action often depends on what has already been found.

The difficulty is that LLM output cannot be trusted in the same way as application policy. A model may return malformed data, select an inappropriate tool, propose an unauthorized target, or react to hostile text contained inside analyzed evidence.

I therefore separated AutoSec-AI into two broad areas.

The first contains AI-facing reasoning. It proposes actions and interprets evidence. The second contains deterministic controls responsible for authorization, tool execution, evidence generation, lifecycle limits, and reporting.

Every proposed action becomes a structured **AgentAction**. An **ActionValidator** checks that action against a **SecurityPolicy** and the registered tool set. Only an allowed action can reach a **SecurityTool** implementation.

I used the same separation when handling results. A scanner finding is evidence produced by a deterministic analyzer. An LLM may later assess that evidence, but the assessment is stored separately and cannot modify the original observation or grant permission for another action.

The current implementation demonstrates this approach through:

- Semgrep-based source-code analysis;
- a small automotive-oriented C rule pack;
- imported-symbol inspection of compiled binaries;
- simulated CAN and UDS behavior;
- diagnostic access-control analysis;
- deterministic bounded UDS fuzzing;
- normalized security evidence;
- structured AI-assisted finding assessment;
- bounded closed-loop investigation;
- terminal and JSON reporting.

At the frozen prototype milestone, the automated test suite contains 193 tests, all passing in the verified development environment.

For me, the important result is not simply that an LLM can call a security tool. The more useful result is that AI reasoning can participate in an investigation while execution authority remains outside the model.

2 Motivation and Problem Statement

2.1 Agentic AI in Security Toolchains

Security investigations rarely consist of a single isolated tool run.

An engineer may begin with source-code analysis, inspect a compiled artifact, look at protocol behavior, correlate findings, and then decide whether another test is justified. This makes cybersecurity an interesting area for agentic AI because previous observations can influence the next investigation step.

However, giving an AI planner access to security tools changes the trust model.

A model might:

- generate malformed output;
- request a tool that should not be available;
- select a target outside the intended scope;
- request excessive parameter values;
- react to adversarial text contained in analyzed data.

The application therefore cannot rely on the model simply “behaving correctly.”

2.2 Core Engineering Problem

The question I used to guide the design was:

How can an agentic AI system participate in an automotive cybersecurity investigation while deterministic software retains control over execution authority?

This led to several separations that appear throughout the implementation:

- proposal versus authorization;
- evidence versus interpretation;
- tool capability versus permission to use that capability;
- investigation feedback versus security policy;
- autonomous reasoning versus lifecycle control.

These boundaries became more important to the project than the choice of any individual scanner or model provider.

3 Objectives and Scope

The main objectives were to:

1. build a structured agentic investigation workflow;
2. treat model-generated actions as untrusted proposals;

3. enforce deterministic authorization before execution;
4. normalize findings from different security domains;
5. keep observed evidence separate from AI inference;
6. allow previous observations to influence later planning;
7. place a hard limit on autonomous investigation;
8. exercise the architecture with automotive-oriented examples;
9. produce reports that preserve evidence provenance.

I deliberately did not try to turn the project into a production vehicle penetration-testing platform. The current scope does not include real vehicle communication, comprehensive static or binary analysis, unrestricted autonomous execution, or production deployment infrastructure.

Keeping that scope narrow made it possible to concentrate on the agentic-security architecture itself.

4 System Architecture

The architecture follows one rule: AI-generated output must cross a deterministic boundary before it can cause a security-sensitive action.

The workflow has four main layers:

1. AI-assisted planning;
2. deterministic authorization;
3. controlled analysis and evidence generation;
4. AI-assisted interpretation and bounded feedback.

Figure 1 shows the high-level flow.

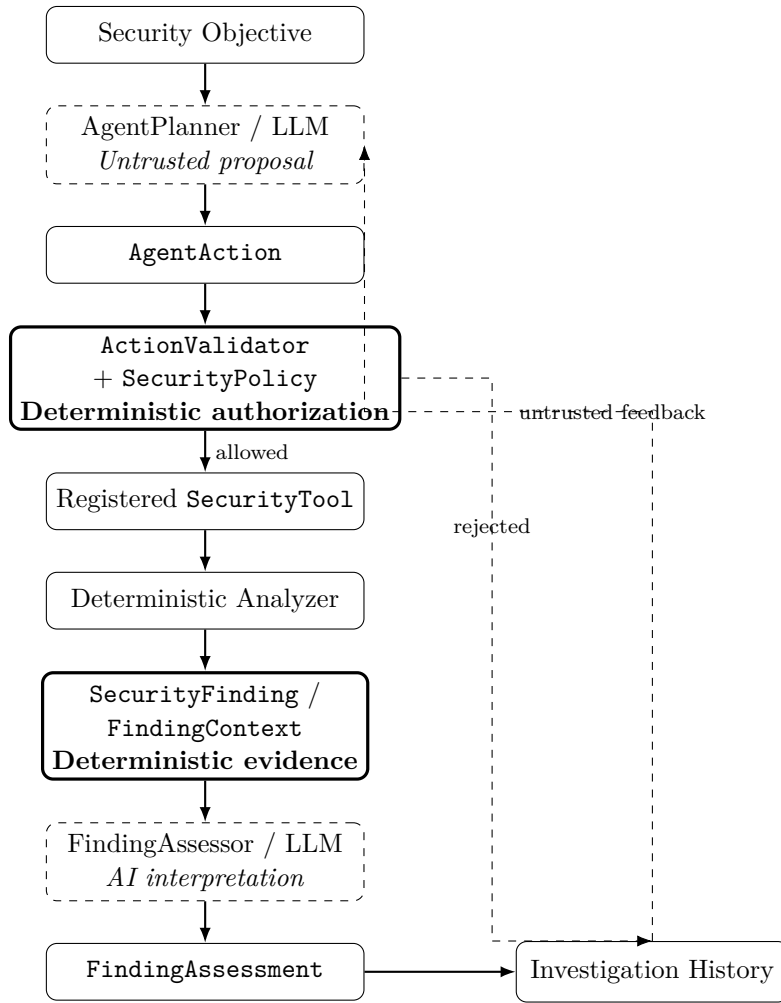


Figure 1: High-level AutoSec-AI investigation architecture. Dashed AI-facing components propose or interpret information; deterministic components retain authorization and evidence-generation responsibilities.

The architecture is intentionally asymmetric. The model has flexibility in reasoning, but the deterministic layer has authority.

5 Agentic AI and Trust-Boundary Design

5.1 LLM Output as a Proposal

The planner receives an objective, a target, and optionally previous investigation feedback. It asks an `LLMClient` for a proposed action and parses the response into an `AgentAction` containing a tool name, target, and parameter mapping.

At that point nothing has been authorized.

This is an important distinction. A syntactically valid action is still only model output.

$$\text{Proposal} \neq \text{Authorization} \quad (1)$$

The planner answers the question “what could I do next?” The validator answers the different question “is this particular action permitted?”

5.2 Proposal and Authorization

Once the planner creates an action, the validator checks the proposed tool, target, and parameters against deterministic application configuration.

If validation fails, the rejection becomes part of the investigation history. The security tool is not executed.

This means that changing a prompt, a previous assessment, or text inside a finding cannot directly expand the permissions available to the model.

5.3 Evidence and Interpretation

I used a similar boundary for analysis results.

A deterministic analyzer may produce a **SecurityFinding** and supporting evidence. An AI-assisted assessor may subsequently classify or explain that finding.

Those outputs are deliberately different objects.

Table 1: Deterministic evidence and AI interpretation have different roles.

	Deterministic layer	AI interpretation layer
Examples	SecurityFinding , NormalizedEvidence , FindingContext	FindingAssessment
Origin	Analyzer, security tool, or controlled simulator	LLM-assisted assessor
Purpose	Record observations and provenance	Interpret supplied evidence
Execution authority	None by itself	None
Exploitability proof	Not necessarily	No

An assessment can be useful without being promoted into evidence that the underlying vulnerability is exploitable.

5.4 Historical Feedback

A closed-loop investigation needs memory of what happened previously.

AutoSec-AI can format previous validation outcomes, tool results, findings, and assessments and supply that information to the planner on the next iteration.

The conceptual loop is:

$$\text{Proposal} \rightarrow \text{Validation} \rightarrow \text{Execution} \rightarrow \text{Evidence} \rightarrow \text{Assessment} \rightarrow \text{Feedback} \quad (2)$$

The feedback is still treated as untrusted input. It cannot modify **SecurityPolicy**, register a new tool, or bypass the validator.

5.5 Bounded Autonomy

I also wanted the investigation to have an explicit stopping condition.

BoundedInvestigationRunner therefore limits the number of new steps in a run. The implementation permits a maximum configuration of ten steps; the controlled source demonstration uses only one.

The modeled termination reasons are:

MAX_STEPS_REACHED The configured number of new investigation steps has completed.

ACTION_REJECTED The validator rejected the proposed action before tool execution.

TOOL_ERROR A tool was executed but returned a **ToolResult** whose status was not **success**.

Unexpected exceptions from execution, result processing, or assessment are not silently converted into a normal tool error. They propagate to the caller.

5.6 Resulting Trust Model

The LLM is allowed to reason, propose, and interpret.

Deterministic code retains control over:

- which tools exist;
- which tools may execute;
- which targets are permitted;
- accepted parameter names and values;
- configured numeric limits;
- trusted rule-pack resolution;
- investigation termination.

The resulting invariant is straightforward:

AI reasoning may influence the investigation, but it does not define its execution authority.

6 Security Policy and Authorization

The authorization layer is the main execution boundary in AutoSec-AI.

Three components are central to it:

- **SecurityPolicy**;
- **ActionValidator**;
- **ToolRegistry**.

6.1 Security Policy

The current **SecurityPolicy** can constrain:

- allowed tools;

- allowed targets;
- exact allowed values for selected parameters;
- numeric upper limits.

Conceptually, an action is allowed only when all required conditions are satisfied:

$$A = T_{\text{registered}} \wedge T_{\text{allowed}} \wedge X_{\text{allowed}} \wedge P_{\text{allowed}} \wedge L_{\text{satisfied}} \quad (3)$$

Here, T represents tool conditions, X target authorization, P parameter authorization, and L configured limits.

6.2 Action Validation

The validator returns a structured decision containing whether the action is allowed, a status code, and a reason.

The execution boundary can be illustrated as:

```

1  action = planner.plan(objective, target, feedback)
2
3  validation = validator.validate(action)
4
5  if not validation.allowed:
6      # Record rejection.
7      # Do not execute a security tool.
8      return rejected_step
9
10 tool = registry.get(action.tool_name)
11 result = tool.execute(
12     action.target,
13     action.parameters
14 )

```

This is an architectural illustration rather than a verbatim copy of the orchestrator implementation.

6.3 Tool Registry

The registry acts as an explicit capability list.

A tool must implement the expected `SecurityTool` abstraction and must be registered before the investigation can use it. Duplicate tool names are rejected, and registration includes a runtime contract check.

This prevents a planner from inventing an executable capability simply by returning an unfamiliar tool name.

6.4 Target Authorization

Allowing a tool does not mean allowing it against every possible target.

The controlled source demonstration, for example, permits Semgrep against the repository's controlled ECU fixture. A different target falls outside that demonstration's authorization scope.

I therefore treat these as separate questions:

Is this tool allowed?

and

Is this tool allowed against this target?

Both must be satisfied.

6.5 Parameter Authorization

Parameters can change tool behavior just as significantly as the tool name itself.

The validator supports exact allowed parameter values and numeric upper limits. Unexpected names or unauthorized values are rejected rather than silently corrected.

The decision does not depend on asking the LLM whether its own parameter choice is safe.

6.6 Trusted Rule-Pack Resolution

Semgrep configuration is a good example of why this matters.

I did not want the planner to supply an arbitrary filesystem path to a Semgrep configuration file. Instead, the planner uses a logical identifier such as:

```
1 automotive
```

Application configuration resolves that identifier through a trusted `RulePackRegistry`.

The resulting path is:

$$\text{Logical ID} \rightarrow \text{Policy Validation} \rightarrow \text{RulePackRegistry} \rightarrow \text{Trusted Rule Path} \quad (4)$$

The planner never needs to choose the rule file's filesystem path.

6.7 External Process Boundary

Some analyzers require external executables. The project uses `ExternalToolRunner` for that boundary.

Commands are invoked using an executable plus an argument list rather than a shell command string:

```
1 subprocess.run(  
2     [executable, *args],  
3     capture_output=True,  
4     text=True,  
5     timeout=timeout,  
6     check=False  
7 )
```

Production project code does not use `shell=True` for these tool invocations.

This reduces shell-command injection exposure, but it should not be confused with process isolation. The prototype does not provide a production sandbox.

6.8 Rejection Is a Normal Outcome

An invalid planner proposal should not crash the security architecture.

When validation rejects an action:

1. the rejection is recorded;
2. the tool is not executed;
3. the bounded run terminates with `ACTION_REJECTED`.

This is important because the security boundary should continue to work even when the planner produces a bad proposal.

7 Source-Code Security Analysis

Source-code analysis is the most complete deterministic analysis path in the current project.

The pipeline is:

AgentAction → SemgrepSecurityTool → SemgrepAnalyzer → Semgrep JSON → SecurityFinding
(5)

7.1 Analyzer Abstraction

Source analyzers implement a common abstraction.

The Semgrep implementation runs the scanner against an authorized target, parses its JSON output, and converts relevant observations into normalized `SecurityFinding` objects.

The normalized representation includes fields such as:

- rule identifier;
- message;
- severity;
- source file and line;
- category;
- CWE identifier;
- originating tool.

Downstream components therefore do not need to understand Semgrep's full native output structure.

7.2 Rule-Pack Design

For the first rule pack, I deliberately kept the analysis narrow.

The logical rule-pack identifier is:

```
automotive
```

and the bundled C rule currently detects:

```
strcpy(...)
```

The rule associates the observation with CWE-120.

The objective was not to build a comprehensive replacement for an established static-analysis ruleset. I needed a small, predictable security finding that could be traced through the complete architecture: scanner execution, normalization, AI assessment, investigation state, and reporting.

7.3 Controlled ECU Fixture

The repository contains an intentionally vulnerable ECU-oriented C fixture used for testing.

Running the trusted rule pack against it produces a normalized finding with information equivalent to:

```
1 Rule: rules.semgrep.autosec-c-unsafe-strcpy
2 Severity: ERROR
3 Category: memory-safety
4 CWE: CWE-120
5 Source: semgrep
6 Message: Unsafe strcpy usage
```

That establishes that the configured pattern was observed. It does not establish remote reachability or practical exploitability.

7.4 Tool Integration

SemgrepSecurityTool adapts the source analyzer to the generic security-tool interface:

$$\text{Orchestrator} \rightarrow \text{SecurityTool} \rightarrow \text{SourceCodeAnalyzer} \rightarrow \text{ExternalToolRunner} \quad (6)$$

The orchestrator therefore does not need Semgrep-specific execution logic.

7.5 Current Processing Limitation

The controlled end-to-end source processor selects the first deterministic finding from the analyzer result and converts it into a `FindingContext`.

That was sufficient for validating the investigation architecture, but it is an intentional limitation. A larger system should retain and correlate multiple findings.

7.6 What the Finding Means

There are three distinct stages:

1. Semgrep observes the configured pattern.
2. AutoSec-AI normalizes the observation.
3. An optional AI assessor interprets it.

None of these stages automatically proves exploitability.

For example, reachability, attacker-controlled input, surrounding bounds checks, compiler behavior, and runtime conditions may still need to be investigated.

8 Binary Security Analysis

I wanted to test whether evidence from a compiled artifact could enter the same investigation model without pretending that the project already contained a full reverse-engineering engine.

The resulting implementation is deliberately small: it performs imported-symbol inspection.

8.1 Imported-Symbol Inspection

The analyzer invokes:

```
1 nm -u <binary-target>
```

and looks for selected unsafe string-copy symbols:

```
1 _strcpy
2 ___strcpy_chk
```

When such a symbol is observed, the analyzer can produce:

```
1 Rule: BIN-MEM-UNSAFE-STRCPY
2 Severity: MEDIUM
3 Category: binary-analysis
4 CWE: CWE-120
5 Source: nm
```

8.2 Meaning of the Evidence

An imported symbol tells us that the binary references the corresponding external function.

It does not tell us:

- whether attacker-controlled data reaches that function;
- the destination-buffer size;
- whether another bounds check exists;
- whether the call is reachable through a relevant path;
- whether practical exploitation is possible.

For that reason I refer to this feature as *imported-symbol inspection*, not binary vulnerability analysis.

8.3 Normalized Binary Evidence

Binary observations can still be useful to the common evidence pipeline:

Binary Artifact $\rightarrow$ Symbol Inspection $\rightarrow$ SecurityFinding $\rightarrow$ NormalizedEvidence $\rightarrow$ FindingContext
(7)

This is primarily an architectural extension point. A more capable binary backend could later replace or supplement the symbol analyzer without requiring the planner, policy, or assessment layers to be redesigned.

9 Automotive CAN/UDS Security Simulation

The automotive part of the project uses an in-memory simulation.

I chose simulation at this stage because the objective was to test the security workflow and evidence model without introducing the operational risk and hardware dependencies of communicating with a real ECU.

9.1 Automotive Data Model

The implementation includes validated models for:

- ECUs;
- classic CAN messages;
- UDS requests;
- UDS responses.

The CAN representation supports 11-bit identifiers and classic CAN payloads of up to eight bytes. It does not implement SocketCAN, ISO-TP, or physical bus communication.

9.2 Simulated ECU

SimulatedECU provides deterministic diagnostic behavior.

The implemented diagnostic example uses UDS ReadDataByIdentifier:

$$SID = 0x22 \quad (8)$$

Two synthetic DIDs are used:

- 0xF190 – synthetic VIN-style data;
- 0xF1A0 – synthetic protected diagnostic data.

These are simulation values and do not originate from a vehicle.

9.3 Diagnostic Access Control

A DiagnosticDataPolicy determines whether protected data may be returned.

Under the secure default configuration, an unauthorized request for the protected DID is denied with:

$$NRC = 0x33 \quad (9)$$

An explicitly insecure configuration can instead return the synthetic protected data.

This gave me a deterministic security condition to analyze:

Protected diagnostic data must not be returned when the simulated policy indicates that access has not been granted.

The analyzer reports a finding only when the observed simulator behavior violates that condition.

9.4 Evidence Path

The resulting evidence flow is:

$$\text{UDS Request} \rightarrow \text{Simulated ECU} \rightarrow \text{UDS Response} \rightarrow \text{Vehicle Analyzer} \rightarrow \text{NormalizedEvidence} \quad (10)$$

The same downstream assessment architecture can therefore consume automotive protocol evidence without being coupled directly to the simulator.

9.5 Simulation Boundary

The current implementation does not contain:

- a physical CAN interface;
- SocketCAN communication;
- ISO-TP transport;
- real ECU diagnostic sessions;
- vehicle-network discovery;
- hardware security testing;
- real diagnostic authentication or SecurityAccess exchange.

The simulator tests framework behavior and selected security logic. It does not measure the security of a production vehicle.

10 Deterministic UDS Fuzzing

The project also includes a small UDS fuzzing component targeting the simulated ReadDataByIdentifier service.

The word *fuzzing* can imply large random campaigns, so it is important to describe what this implementation actually does. It uses a small deterministic mutation set designed for reproducible testing.

10.1 Mutation Cases

The implemented cases include:

1. an empty request payload;
2. a truncated DID representation;
3. a valid DID representation;
4. a request containing an additional byte;
5. an eight-byte classic-CAN-sized payload.

The same configured run produces the same ordered inputs.

10.2 Bounded Execution

The configuration permits at most 32 cases. The current implementation uses only the small fixed corpus above.

It does not use:

- infinite mutation loops;
- high-volume random generation;
- parallel workers;
- automatic retries;
- network communication;
- physical vehicle interfaces.

10.3 Generation and Analysis Are Separate

The fuzzer generates a case, submits it to the simulator, and records the response.

A separate `UDSFuzzResponseAnalyzer` decides whether the response matches a security-relevant deterministic predicate.

$$\text{Mutation} \rightarrow \text{Fuzzer} \rightarrow \text{Simulator} \rightarrow \text{Response Evidence} \rightarrow \text{Analyzer} \quad (11)$$

I kept these responsibilities separate because the component producing test inputs should not automatically become the authority that labels its own output a vulnerability.

10.4 Why Start with Deterministic Fuzzing?

There were three practical reasons.

First, deterministic cases are easy to reproduce in tests.

Second, a fixed corpus fits the project's broader use of explicit execution bounds.

Third, it let me validate the orchestration and evidence architecture without making unsupported claims about protocol coverage.

A future fuzzer could use richer mutation or coverage feedback, but that is outside the current implementation.

11 Evidence Normalization and AI Assessment

Once several analyzers existed, a second design problem became apparent: each tool naturally describes evidence differently.

Semgrep output, an imported binary symbol, a diagnostic response, and a fuzz result do not share the same native format.

I introduced a normalized evidence layer so that later investigation components would not need tool-specific logic.

11.1 SecurityFinding

A `SecurityFinding` records a deterministic analyzer finding using normalized fields such as:

- rule identifier;
- message;
- severity;
- source location where applicable;
- category;
- CWE where applicable;
- source tool.

Normalization does not reinterpret the finding. It changes the representation, not the meaning.

11.2 NormalizedEvidence and FindingContext

Binary, vehicle, and fuzzing adapters can convert their observations into `NormalizedEvidence`.

A `FindingContext` combines a finding with supporting normalized evidence.

$$\text{Tool Observation} \rightarrow \text{NormalizedEvidence} \rightarrow \text{FindingContext} \quad (12)$$

This gives the assessor a common interface while preserving provenance.

11.3 AI-Assisted Assessment

The assessor can turn a `FindingContext` into a structured `FindingAssessment` containing:

- classification;
- confidence;
- rationale;
- impact;

- recommendations.

The supported classification values are:

- `confirmed_vulnerability`;
- `likely_vulnerability`;
- `needs_review`;
- `likely_false_positive`.

Confidence is similarly restricted to the configured structured values.

The LLM response must match the expected structure. Malformed or unexpected output is rejected rather than silently accepted.

11.4 Evidence, Assessment, and Authority

These concepts should not collapse into one another:

$$\text{Evidence} \neq \text{Assessment} \neq \text{Authorization} \quad (13)$$

If Semgrep reports an unsafe string operation, that is the deterministic observation.

The LLM may explain why it deserves attention, but its explanation does not rewrite the scanner result and does not authorize another tool.

11.5 Prompt-Injection Considerations

Analyzed artifacts may themselves contain hostile text.

Finding context is therefore serialized in a deterministic structured form before being supplied to the assessor. Finding and evidence content are identified as untrusted analysis data.

The same rule applies to historical feedback. Previous observations may influence reasoning, but they cannot change the policy boundary.

12 Closed-Loop Investigation

Individual tool execution becomes more interesting when the result of one step can influence the next.

AutoSec-AI supports that behavior through immutable investigation history and a bounded runner.

12.1 Investigation State

`InvestigationState` contains the objective, target, and ordered history of `InvestigationStep` objects.

A step can record:

- proposed action;
- validation outcome and status;

- tool result;
- finding context;
- AI assessment.

New history is appended through updated state rather than by modifying earlier step records.

12.2 One Orchestration Step

`InvestigationOrchestrator` performs one investigation step:

1. ask the planner for a proposal;
2. validate the proposal;
3. record and stop if it is rejected;
4. otherwise execute the registered tool once;
5. process a successful result when a processor exists;
6. assess generated finding context when an assessor exists;
7. record the completed step.

Keeping a single step separate from the loop makes the lifecycle easier to test.

12.3 Bounded Runner

`BoundedInvestigationRunner` repeats the one-step operation subject to the configured maximum.

There is no unbounded `while` loop, recursive autonomous execution, or automatic retry loop in the runner.

The configured maximum refers to new investigation steps in that run.

12.4 Feedback Does Not Increase Authority

Historical information can be formatted and supplied to the next planner invocation:

$$S_n \rightarrow P_{n+1} \rightarrow V_{n+1} \rightarrow E_{n+1} \rightarrow S_{n+1} \quad (14)$$

where S is state, P proposal, V validation, and E execution and evidence.

The key point is:

$$\text{More Context} \not\Rightarrow \text{More Permission} \quad (15)$$

Every newly proposed action must pass authorization independently.

13 End-to-End Automotive Source Investigation

The controlled source demonstration connects the major pieces into one reproducible run.

I intentionally use deterministic mock model responses in this demo. That keeps the demonstration reproducible and means that a reviewer can run it without an API key or external model connection.

13.1 Composition

The demonstration combines:

- `MockLLMClient` for deterministic planning;
- `AgentPlanner`;
- `ToolRegistry`;
- `SemgrepSecurityTool`;
- `SemgrepAnalyzer`;
- `RulePackRegistry`;
- `SecurityPolicy`;
- `ActionValidator`;
- `SourceFindingResultProcessor`;
- `LLMFindingAssessor` using a deterministic mock client;
- `InvestigationOrchestrator`;
- `BoundedInvestigationRunner`.

The runner is configured for one step.

13.2 Execution

During the demonstration:

1. the planner proposes `Semgrep`;
2. the target is the controlled ECU source fixture;
3. the logical rule pack is `automotive`;
4. deterministic policy validates the action;
5. `Semgrep` executes using the trusted rule mapping;
6. the `strcpy` rule produces a CWE-120 finding;
7. the result becomes a `FindingContext`;
8. the mock assessor returns `needs_review`;
9. the step is recorded;
10. the one-step bound produces `MAX_STEPS_REACHED`.

13.3 Representative Output

```
1  SIMULATED / CONTROLLED AUTOMOTIVE SECURITY DEMONSTRATION
2  AUTOSEC-AI SECURITY INVESTIGATION
3
4  Objective:
5  Assess the controlled automotive ECU source fixture.
6
7  Target:
8  tests/fixtures/ecu_vulnerable.c
9
10 Termination: MAX_STEPS_REACHED
11 Steps: 1
12
13 Proposed tool: semgrep
14 Parameters: {"rule_pack": "automotive"}
15
16 Authorization: ALLOWED
17 Validation: VALID
18 Tool result: success
19
20 Deterministic finding
21 Rule: rules.semgrep.autosec-c-unsafe-strcpy
22 Severity: ERROR
23 Category: memory-safety
24 CWE: CWE-120
25 Source: semgrep
26 Message: Unsafe strcpy usage
27
28 AI assessment
29 Classification: needs_review
30 Confidence: medium
```

13.4 What This Demonstrates

The important part of the result is the complete path.

A planner proposal crossed deterministic authorization, a real scanner generated evidence, that evidence was normalized, an assessor interpreted it, and the investigation terminated at its configured bound.

MAX_STEPS_REACHED is therefore expected in this demonstration; it is not an error condition.

The result does not prove that the controlled fixture contains a practically exploitable automotive vulnerability. It proves that the implemented investigation components can work together while preserving their trust boundaries.

14 Security Evaluation and Testing

Testing became an important part of this project because many of the most important requirements are negative requirements.

It is not enough to verify that an authorized tool can run. I also need confidence that a rejected action does *not* run, malformed model output does not become an action, and investigation history does not silently expand permissions.

14.1 Automated Test Suite

At the frozen prototype milestone:

193 tests — 193 passing

in the verified development environment.

The suite covers areas including:

- tool registration;
- policy enforcement;
- unauthorized tool and target rejection;
- parameter validation;
- numeric limits;
- malformed planner responses;
- malformed assessment responses;
- prompt-injection handling;
- rejected-action non-execution;
- bounded investigation;
- tool-reported failure;
- evidence normalization and provenance;
- reporting;
- the end-to-end source demonstration.

I do not report a code-coverage percentage because a formal coverage measurement is not part of this milestone.

14.2 Investigation Evaluation

`InvestigationEvaluation` summarizes observable execution behavior, including:

- total steps;
- allowed and rejected actions;
- executed actions;
- successful and failed tool executions;
- deterministic findings;
- AI assessments;
- termination reason.

These are execution metrics.

They are not CVSS scores, vulnerability-risk scores, exploitability probabilities, or measurements of AI accuracy.

14.3 Security-Oriented Testing

Some particularly important properties are:

- rejected actions never execute their requested tool;
- malformed LLM output cannot silently become an authorized action;
- an unknown tool cannot be created through model output;
- unauthorized rule-pack values are rejected;
- historical assessment text cannot bypass policy;
- autonomous execution cannot exceed its configured bound.

For an agentic security tool, testing what must *not* happen is as important as testing the successful path.

15 Threat Model and Security Invariants

The threat model focuses on the boundary created when AI-generated content participates in security-tool orchestration.

This is a local engineering prototype. It is not a network-facing multi-user service and it does not connect to a real vehicle.

15.1 Protected Assets

The main assets considered are:

- execution authority;
- approved targets;
- approved tool parameters;
- trusted rule-pack mappings;
- deterministic evidence;
- investigation history;
- policy configuration;
- report presentation integrity.

15.2 Threats and Controls

Table 2: Selected threats and controls implemented in the prototype.

Threat	Implemented control
Unauthorized tool	Registry lookup and explicit allowed-tool policy.
Unauthorized target	Explicit target allow-list.
Unexpected parameter	Parameter-name and value validation.
Excessive numeric value	Configured deterministic upper limit.
Arbitrary Semgrep config	Logical rule-pack identifier resolved through trusted application configuration.
Shell injection	External process invoked using an argument list without <code>shell=True</code> .
Prompt injection in evidence	Structured serialization and explicit untrusted-data framing.
Feedback changes authority	Each new proposal independently crosses the validator.
Unbounded investigation	Hard step bound with a maximum configuration of ten.
Rejected action executes	Rejection is recorded without calling tool execution.
Terminal control content	Presentation layer sanitizes ANSI and other control characters.
Assessment confused with evidence	Separate finding/context and assessment models.

15.3 Security Invariants

The implementation is designed to maintain the following invariants:

1. No proposed action executes before deterministic validation.
2. LLM output never directly grants execution authority.
3. Tool, target, and parameter authorization remain outside the LLM.
4. AI assessment is not deterministic proof of exploitability.
5. Every autonomous iteration crosses validation independently.
6. Autonomous execution is bounded.
7. Trusted rule paths come from application configuration, not planner-supplied filesystem paths.
8. Rejected actions do not execute security tools.
9. Historical findings and assessments cannot modify policy.
10. A returned non-success tool result terminates the bounded run with `TOOL_ERROR`.

15.4 Residual Risk

Several issues would need more work before a production deployment.

External tools are not sandboxed. `SecurityPolicy` is currently a mutable object if another component retains its reference. Application composition and rule-pack mappings are trusted.

External scanner output is not independently verified. Unexpected implementation exceptions propagate to the caller.

The default demonstration also uses deterministic mock model responses, so it does not attempt to evaluate the behavior of a live provider model under adversarial conditions.

There is currently no authentication, persistent investigation service, or multi-user authorization model.

16 Limitations

The current scope is deliberately narrower than the architecture could eventually support.

- **Vehicle interaction is simulated.** There is no physical CAN interface, ISO-TP stack, ECU connection, or real vehicle communication.
- **The source rule pack is small.** The bundled automotive example focuses on a controlled `strcpy`/CWE-120 case rather than broad automotive static analysis.
- **The source demo processes one finding.** The demonstration selects the first deterministic scanner finding rather than correlating an arbitrary set.
- **Binary inspection is intentionally limited.** It examines selected imported symbols with `nm -u`; there is no disassembly, call-graph reconstruction, taint analysis, or dynamic instrumentation.
- **UDS fuzzing uses a fixed corpus.** It is deterministic and simulator-only, not a random, coverage-guided, high-volume, or real-vehicle fuzzing system.
- **The default demo uses mock model responses.** This improves reproducibility but does not evaluate live-model variability.
- **Findings do not prove exploitability.** Deterministic observations and AI assessments may justify further investigation, but neither automatically establishes practical exploitation.
- **External execution is not sandboxed.** Application-level authorization exists, but the prototype does not provide production-grade process isolation.
- **There is no persistent distributed service.** The current implementation does not provide databases, distributed workers, production authentication, or multi-user access control.

These are boundaries of the current implementation rather than hidden assumptions about capability.

17 Future Work

The next useful extensions are fairly clear from the limitations above.

17.1 Source Analysis

The rule-pack model can support additional automotive-oriented rules for memory safety, diagnostic interfaces, unsafe parsing, and cryptographic misuse.

I would also replace the first-finding source processor with multi-finding collection and correlation.

17.2 Binary Analysis

A dedicated reverse-engineering backend could provide richer evidence such as:

- function metadata;
- imported and exported functions;
- call relationships;
- selected control-flow information;
- strings and constant references;
- security-relevant binary properties.

The normalized evidence boundary means such a backend could be added without making the AI assessor responsible for parsing raw tool output.

17.3 Controlled Vehicle Interfaces

The simulator could eventually be complemented by explicitly authorized laboratory adapters for:

- SocketCAN;
- ISO-TP;
- additional UDS services;
- diagnostic-session handling;
- controlled SecurityAccess experiments;
- virtual or hardware-in-the-loop ECU environments.

Moving from simulation to real communication would also require stronger target controls, rate limiting, operational safeguards, and authorization.

17.4 Fuzzing

The deterministic fuzzing foundation could be extended with richer mutation strategies, protocol-state awareness, corpus management, and coverage feedback.

The separation between generation, response evidence, and analysis should remain even if the fuzzing engine becomes more capable.

17.5 Execution Isolation

A production-oriented implementation would need stronger isolation, potentially including:

- sandboxed workers;
- filesystem isolation;
- resource quotas;
- explicit network restrictions;

- stronger immutable policy configuration;
- auditable capability assignment.

17.6 Analyst Interface

An analyst-facing interface could make investigation history easier to review by showing authorization decisions, deterministic evidence, AI interpretation, and termination state separately.

Persistent artifacts would also make investigations easier to reproduce and compare.

17.7 Evaluation

A larger controlled corpus would make it possible to evaluate analyzer behavior and the usefulness of AI-assisted triage more systematically.

Any future metric for model quality should remain separate from authorization and deterministic security evidence.

18 Conclusion

AutoSec-AI was built to explore the boundary between AI reasoning and security-sensitive execution.

The project convinced me that these do not need to be the same thing.

An LLM can propose an investigation step, interpret a finding, and use previous observations to reason about what might be useful next. None of those capabilities requires the model itself to decide what it is allowed to execute.

In AutoSec-AI, that authority stays with deterministic application logic. Tools must be registered, targets and parameters must satisfy policy, and every new proposal crosses the same authorization boundary. Findings are kept separate from AI interpretation, and autonomous execution has a defined upper bound.

The prototype exercises these ideas through Semgrep source analysis, limited binary-symbol inspection, simulated CAN/UDS security behavior, deterministic UDS fuzzing, evidence normalization, AI-assisted assessment, closed-loop investigation, and reporting.

The controlled source demonstration provides the clearest example. The planner proposes Semgrep, policy authorizes the action, the real scanner produces a deterministic CWE-120 observation, the finding is normalized, the assessor interprets it, and the investigation terminates at its configured bound.

There is still substantial work between this prototype and a production automotive security platform. Real vehicle communication, richer binary analysis, larger rule sets, more capable fuzzing, execution isolation, and persistent investigation infrastructure are all outside the current milestone.

That limitation is useful rather than accidental. It keeps the main engineering result clear:

The model proposes. Deterministic policy decides.

Agentic AI can add flexibility to a cybersecurity investigation without being given unrestricted control of the security tools it coordinates.

A Prototype Environment

The prototype was developed and verified using the following software environment:

- Python 3.11.9;
- Semgrep 1.177.0;
- OpenAI Python SDK 3.13.0 for optional provider integration;
- pytest for automated testing;
- Git for version control;
- Clang 14.0.0 and standard binary utilities including `nm`;
- macOS development environment.

Detailed personal hardware information is not relevant to reproducing the architecture and is intentionally omitted.

At the frozen prototype milestone:

193 tests — 193 passing

in the verified development environment.

This test result is an implementation milestone, not a security certification or production-readiness claim.

B Reproducible Demonstration

After installing the project and development dependencies according to the repository README, the automated tests can be executed from the repository root with:

```
python -m pytest
```

The controlled source investigation can be run with:

```
python -m autosec_ai.cli source-demo
```

and its JSON representation with:

```
python -m autosec_ai.cli source-demo --json
```

The default demonstration uses controlled repository fixtures and deterministic mock LLM responses. It therefore does not require an API key, vehicle interface, or external model connection.

Live model integration through the optional provider client is configured separately.

API credentials are not stored in the repository.

C Project Repository

The implementation, automated tests, architecture documentation, threat model, controlled fixtures, Semgrep rules, and demonstration instructions are maintained at:

<https://github.com/trivendraku/hare/AutoSec-AI>

The repository README is the primary entry point for installation and execution instructions. This report documents the design reasoning, security boundaries, implemented capabilities, evaluation, and current limitations.